

Fiche Code — Data Science en Python

Le compagnon « pratique » de la fiche révision : comment chaque notion s'écrit vraiment en code

On reprend les **mêmes notions**, mais cette fois en **code Python qui tourne**. Pour chaque point : une image simple, le code, l'explication ligne par ligne, et le **vrai résultat** obtenu. Tous les exemples ont été exécutés avec **scikit-learn**, la boîte à outils standard du Data Scientist.

Couleurs : **bleu** = l'image simple • **vert** = exemple • **gris** = résultat affiché • **orange** = à retenir.

0. Mise en route : les outils et le jeu de données

On installe une fois les bibliothèques, puis on importe ce qu'il faut. **scikit-learn** contient les modèles ; **pandas** gère les tableaux ; **numpy** fait les calculs.

```
pip install scikit-learn pandas numpy # une seule fois, dans le terminal
```

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
```

On utilisera un petit jeu « jouet » facile à imaginer : **200 fruits**. Pour chacun, on connaît son **poids** (g) et sa **taille** (cm). L'étiquette est **0 = pomme** ou **1 = orange**.

```
poids = [150, 205, 148, 198, ...] # une valeur par fruit
taille = [7.0, 8.6, 6.8, 8.4, ...]
X = np.c_[poids, taille] # X = les caractéristiques (2 colonnes)
y = [0, 1, 0, 1, ...] # y = la bonne réponse (pomme/orange)
```

```
# On coupe en 2 : entraînement (apprendre) et test (noter sur du jamais-vu)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, random_state=42)
```

L'image simple : On donne 150 fruits avec leur nom à l'élève pour qu'il apprenne (train), et on garde 50 fruits cachés pour l'interroger après (test).

Résultat : train (150, 2) test (50, 2)

À retenir : **X** = ce qu'on connaît (les colonnes). **y** = ce qu'on veut deviner. On garde TOUJOURS un jeu de test à part.

1. Classification supervisée

Schéma identique pour tous les modèles scikit-learn : **.fit(Xtr, ytr)** pour apprendre, **.predict(Xte)** pour deviner, **.score(Xte, yte)** pour la note (accuracy).

1.1 Régression logistique

```
from sklearn.linear_model import LogisticRegression
clf = LogisticRegression().fit(Xtr, ytr) # apprend
print(clf.score(Xte, yte)) # note sur le test
print(clf.predict_proba(Xte[:1]).round(2)) # probabilités du 1er fruit
```

Ligne par ligne : on crée le modèle, **.fit** lui fait apprendre, **.score** donne l'accuracy, et **predict_proba** donne la **probabilité** de chaque classe.

Résultat : 1.0 [[0.91 0.09]] # 91% pomme, 9% orange

L'image simple : La machine ne dit pas juste « pomme » : elle dit « 91 % pomme ». Au-dessus de 50 %, elle tranche « pomme ».

1.2 Forêt aléatoire (+ importance des variables)

```
from sklearn.ensemble import RandomForestClassifier
rf = RandomForestClassifier(n_estimators=100, random_state=0).fit(Xtr, ytr)
print(rf.score(Xte, yte))
print(rf.feature_importances_.round(2)) # quelle variable compte le plus
```

`n_estimators=100` = **100 arbres** qui votent. **feature_importances_** dit le poids de chaque variable (ici poids vs taille).

Résultat : 0.98 [0.55 0.45] # le poids compte 55%, la taille 45%

Exemple concret : Pour un prêt bancaire, `feature_importances_` te dirait par exemple que « le salaire » pèse plus que « l'âge » — utile pour expliquer la décision.

1.3 KNN et l'effet du nombre de voisins k

```
from sklearn.neighbors import KNeighborsClassifier
for k in [1, 5, 15]:
    knn = KNeighborsClassifier(n_neighbors=k).fit(Xtr, ytr)
    print(k, round(knn.score(Xtr, ytr), 2), round(knn.score(Xte, yte), 2))
```

On essaie plusieurs k et on compare la note **train** et la note **test**.

Résultat : k=1 train=1.00 test=0.96 <- par coeur (surapprentissage) k=5 train=0.95 test=0.90 k=15 train=0.93 test=0.92 <- plus lisse

À retenir : Quand **train = 1.00** mais test plus bas, c'est le signe d'un **surapprentissage**. Augmenter k lisse la décision.

1.4 SVM

```
from sklearn.svm import SVC
svm = SVC(kernel="rbf").fit(Xtr, ytr) # rbf = frontière courbée
print(svm.score(Xte, yte))
```

Résultat : 0.98

2. Clustering (non supervisé)

Ici **pas de y** : on donne seulement X, et l'algorithme trouve les groupes tout seul. On lit le résultat dans **.labels_** (le numéro de groupe de chaque point).

2.1 K-means + silhouette + méthode du coude

```
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

km = KMeans(n_clusters=2, n_init=10, random_state=0).fit(X)
print(km.labels_[:8]) # groupe de chaque fruit
print(round(silhouette_score(X, km.labels_), 2))

for k in [2, 3, 4]: # méthode du coude
    m = KMeans(n_clusters=k, n_init=10).fit(X)
    print(k, round(m.inertia_))
```

silhouette (entre -1 et 1) : **plus c'est haut, mieux les groupes sont séparés**. **inertia_** baisse quand k monte ; on choisit le k au « coude » de la courbe.

Résultat : [1 1 1 0 1 1 1 1] 0.6 2 56044 3 24195 4 16110

L'image simple : La silhouette, c'est : « es-tu bien rangé dans ta boîte, ou à cheval entre deux ? ». 0,6 = plutôt bien rangé.

2.2 Clustering hiérarchique (CAH)

```
from sklearn.cluster import AgglomerativeClustering
ac = AgglomerativeClustering(n_clusters=2).fit(X)
print(ac.labels_[:6])
```

Résultat : [0 0 0 0 0 0]

2.3 DBSCAN (par densité, détecte le bruit)

```
from sklearn.cluster import DBSCAN
db = DBSCAN(eps=1.0, min_samples=5).fit(X)
labels = db.labels_
# eps = rayon du voisinage ; les points isolés reçoivent l'étiquette -1 (bruit)
n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
print(n_clusters, "groupes,", list(labels).count(-1), "points de bruit")
```

Sur des données à 3 groupes serrés + des points dispersés, DBSCAN retrouve les groupes et marque les solitaires comme **bruit (-1)**, sans qu'on lui donne le nombre de groupes.

Résultat : 3 groupes, 9 points de bruit

3. Régression (prédire un nombre)

Nouvel exemple : prédire le **prix d'un appartement** à partir de sa **surface** (m²).

3.1 Régression linéaire

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

lin = LinearRegression().fit(Xtr2, ytr2) # Xtr2 = surface, ytr2 = prix
pred = lin.predict(Xte2)
print(round(lin.coef_[0]), round(lin.intercept_)) # pente, ordonnée
print(round(mean_absolute_error(yte2, pred))) # MAE
print(round(mean_squared_error(yte2, pred) ** 0.5)) # RMSE
print(round(r2_score(yte2, pred), 3)) # R2
```

La droite apprise est **prix $\approx$ 3046 $\times$ surface + 48901**. **coef_** = le prix par m². MAE/RMSE = l'erreur moyenne en €. R² proche de 1 = très bon.

Résultat : 3046 48901 12771 # MAE : on se trompe de ~12 800 € en moyenne 15832 #
RMSE 0.973 # R2 : le modèle explique 97% du prix

L'image simple : Chaque m² ajoute ~3046 € au prix. Un appart « de base » (0 m²) vaudrait 48 901 € : c'est l'ordonnée à l'origine.

3.2 Ridge / Lasso (anti-triche) et Gradient Boosting

```
from sklearn.linear_model import Ridge, Lasso
from sklearn.ensemble import GradientBoostingRegressor

Ridge(alpha=1.0).fit(Xtr2, ytr2) # alpha = force de la pénalité
Lasso(alpha=100.0).fit(Xtr2, ytr2) # Lasso peut mettre des coeffs à 0
GradientBoostingRegressor().fit(Xtr2, ytr2) # arbres en série (non linéaire)
```

Résultat : Ridge R2 = 0.957 Lasso R2 = 0.957 GradientBoosting R2 = 0.906

À retenir : **alpha** contrôle la régularisation : plus il est grand, plus on bride le modèle pour éviter le surapprentissage.

4. Évaluer un modèle

4.1 Classification : matrice de confusion et métriques

```
from sklearn.metrics import (confusion_matrix, accuracy_score,
```

```

precision_score, recall_score, f1_score)
pred = rf.predict(Xte)
print(confusion_matrix(yte, pred))
print(accuracy_score(yte, pred), precision_score(yte, pred),
      recall_score(yte, pred), f1_score(yte, pred))

```

La matrice : ligne = vérité, colonne = prédiction. La diagonale = les bonnes réponses.

Résultat : [[26 1] # 26 pommes OK, 1 pomme prise pour une orange (FP) [0 23]] # 0
orange ratée (FN), 23 oranges OK acc=0.98 prec=0.96 rec=1.0 f1=0.98

Astuce : **classification_report(yte, pred)** affiche précision/rappel/F1 d'un coup pour chaque classe.

4.2 Régression : MAE, RMSE, R²

Mêmes fonctions que plus haut : **mean_absolute_error**, **mean_squared_error** (mettre ** 0.5 pour la RMSE), **r2_score**.

4.3 Clustering : silhouette

Comme il n'y a pas de vérité, on juge la qualité des groupes avec **silhouette_score(X, labels)** (vu en 2.1).

5. Repérer et éviter le surapprentissage

5.1 Comparer train et test

```

from sklearn.tree import DecisionTreeClassifier
deep = DecisionTreeClassifier(max_depth=None).fit(Xtr, ytr) # sans limite
shallow = DecisionTreeClassifier(max_depth=2).fit(Xtr, ytr) # bridé
print(deep.score(Xtr, ytr), deep.score(Xte, yte))
print(shallow.score(Xtr, ytr), shallow.score(Xte, yte))

```

Résultat : 1.00 0.98 # arbre profond : parfait sur train (suspect) 0.95 0.96 # arbre limité :
train et test proches (sain)

L'image simple : L'arbre sans limite apprend par coeur (100% sur train). L'arbre bridé comprend l'idée générale : ses notes train et test se ressemblent.

5.2 La validation croisée

```

from sklearn.model_selection import cross_val_score
scores = cross_val_score(rf, X, y, cv=5) # 5 découpes différentes
print(scores.round(2), "moyenne", round(scores.mean(), 2))

```

On teste le modèle sur **5 découpes différentes** et on regarde la moyenne : bien plus fiable qu'un seul test.

Résultat : [0.95 0.95 1. 0.98 0.92] moyenne 0.96

À retenir : Parades : **max_depth** (arbres), **alpha** (Ridge/Lasso), **cross_val_score**, et plus de données.

6. Choisir la métrique : jouer sur le seuil

Par défaut, le seuil de décision est 0.5. En le **baissant**, on attrape plus de positifs (rappel ↑) mais on fait plus de fausses alertes (précision ↓). C'est le réglage clé pour une **détection de bombes** (on veut un rappel maximal).

```

proba = clf.predict_proba(Xte)[:, 1] # proba d'etre "positif"
for seuil in [0.5, 0.3, 0.2]:
    pred = (proba >= seuil).astype(int)
    print(seuil, recall_score(yte, pred), precision_score(yte, pred))

```

(Ici sur un jeu plus difficile, avec une classe rare — comme une vraie détection.)

Résultat : seuil 0.5 : rappel 0.58 precision 0.90 seuil 0.3 : rappel 0.74 precision 0.79 seuil 0.2 :
rappel 0.84 precision 0.65

L'image simple : Baisser le seuil = être plus méfiant : tu sonnes l'alarme plus souvent. Tu rates moins de vrais dangers (rappel ↑), mais tu cries au loup plus souvent (précision ↓).

7. Réduire la dimension : PCA

```
from sklearn.decomposition import PCA
pca = PCA(n_components=2).fit(X10) # X10 : 10 colonnes
Xreduit = pca.transform(X10) # devient 2 colonnes
print(pca.explained_variance_ratio_.round(2),
      round(pca.explained_variance_ratio_.sum(), 2))
```

La PCA **résume plein de colonnes en quelques axes**. `explained_variance_ratio_` dit quelle part d'information chaque axe garde.

Résultat : [0.97 0.00] total 0.98 # 2 axes suffisent à garder 98% de l'info

L'image simple : Au lieu de décrire un objet avec 10 mesures qui se répètent, on garde 2 « super-mesures » qui résument presque tout.

8. Détecter les valeurs aberrantes (IQR)

```
data = np.array([10, 12, 11, 13, 12, 250, 11, 10])
q1, q3 = np.percentile(data, [25, 75])
iqr = q3 - q1
low, high = q1 - 1.5 * iqr, q3 + 1.5 * iqr
print(data[(data < low) | (data > high)]) # les valeurs hors bornes
```

La méthode de l'écart interquartile : tout ce qui sort de **[Q1-1,5·IQR ; Q3+1,5·IQR]** est suspect.

Résultat : bornes [8.5 ; 14.5] -> aberrant : [250]

L'image simple : Dans une liste de tailles autour de 11-13, un « 250 » saute aux yeux : c'est une erreur de saisie à corriger.

9. Mettre à la même échelle

```
from sklearn.preprocessing import StandardScaler, MinMaxScaler
scaler = StandardScaler().fit(Xtr) # apprend moyenne/ecart-type sur le train
Xtr_scaled = scaler.transform(Xtr) # applique
print(Xtr.mean(0).round(1), "->", Xtr_scaled.mean(0).round(2))
```

StandardScaler ramène chaque colonne à **moyenne 0, écart-type 1**. MinMaxScaler mettrait tout entre 0 et 1.

Résultat : [177.6 7.7] -> [-0. 0.] # tout est recentré

À retenir : On **fit le scaler sur le train uniquement**, puis on l'applique au test (sinon on triche). Le plus propre : un **Pipeline**.

```
from sklearn.pipeline import make_pipeline
pipe = make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=5))
pipe.fit(Xtr, ytr) # scaling + modèle enchaînés automatiquement
print(pipe.score(Xte, yte))
```

Résultat : 0.96

10. Gérer les données manquantes

```
import pandas as pd
df = pd.DataFrame({"age": [25, None, 40, 30],
                  "salaire": [3000, 3500, None, 2800]})
print(df.isna().sum()) # combien de trous par colonne
```

```
from sklearn.impute import SimpleImputer
imp = SimpleImputer(strategy="median") # remplit avec la médiane
print(imp.fit_transform(df))
```

.isna().sum() compte les trous ; **SimpleImputer** les remplit (médiane/moyenne/mode) ; **KNNImputer** fait plus fin (à partir des lignes ressemblantes).

```
Résultat : age 1 / salaire 1 [[ 25 3000] [ 30 3500] # le trou d'age rempli par 30 (médiane) [ 40 3000] [ 30 2800]]
```

11. Visualiser la variable cible

```
import matplotlib.pyplot as plt
import seaborn as sns

# cible catégorielle : combien de pommes vs oranges
sns.countplot(x=y); plt.show()

# variable continue : distribution du prix
plt.hist(prix, bins=20); plt.show()

# relation deux variables : nuage de points coloré par classe
plt.scatter(poids, taille, c=y); plt.show()
```

Cible **catégorie** → barres (**countplot**). Cible **nombre** → **hist()**. Relations → **scatter()**. **plt.show()** affiche le graphe.

12. Python pur : lambda, map, filter, Two Sum

12.1 Carrés des nombres pairs (Q18)

```
liste = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
evenNumbers = list(map(lambda x: x**2, filter(lambda x: x % 2 == 0, liste)))
print(evenNumbers)
```

filter garde les pairs ($x \% 2 == 0$), **map** les met au carré, **list** matérialise. Version équivalente plus lisible : `[x**2 for x in liste if x%2==0]`.

```
Résultat : [4, 16, 36, 64, 100]
```

12.2 Two Sum (Q19)

```
def two_sum(nums, target):
    vus = {} # valeur déjà vue -> son indice
    for i, n in enumerate(nums):
        if target - n in vus: # le complément est-il déjà connu ?
            return [vus[target - n], i]
        vus[n] = i

print(two_sum([2, 7, 11, 15], 9)) # [0, 1]
print(two_sum([3, 2, 4], 6)) # [1, 2]
```

L'image simple : Tu veux deux cartes qui font 9. Tu tiens un « 2 ». Tu regardes dans ton carnet si tu as déjà vu un « 7 ». Sinon, tu notes « 2 » et tu continues. Une seule passe : rapide.

```
Résultat : [0, 1] [1, 2]
```

13. Bonus : un mini-projet de bout en bout

Tout s'enchaîne dans l'ordre d'un vrai projet : charger → couper → (scaler) → entraîner → évaluer.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report

# 1) données : X = caractéristiques, y = cible
```

```
# 2) couper
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, random_state=42)
# 3) + 4) scaling et modèle enchaînés
model = make_pipeline(StandardScaler(), RandomForestClassifier())
model.fit(Xtr, ytr)
# 5) évaluer sur le jamais-vu
print(classification_report(yte, model.predict(Xte)))
```

À retenir : C'est le squelette de presque tous tes projets : **couper, entraîner, évaluer sur le test**. Si tu maîtrises ça, tu es prêt comme Data Scientist junior.

Entraîne-toi à retaper ces blocs sans regarder, en changeant juste le modèle (LogisticRegression, SVC, KNN...). Le reste du code ne bouge pas. Bon courage !